

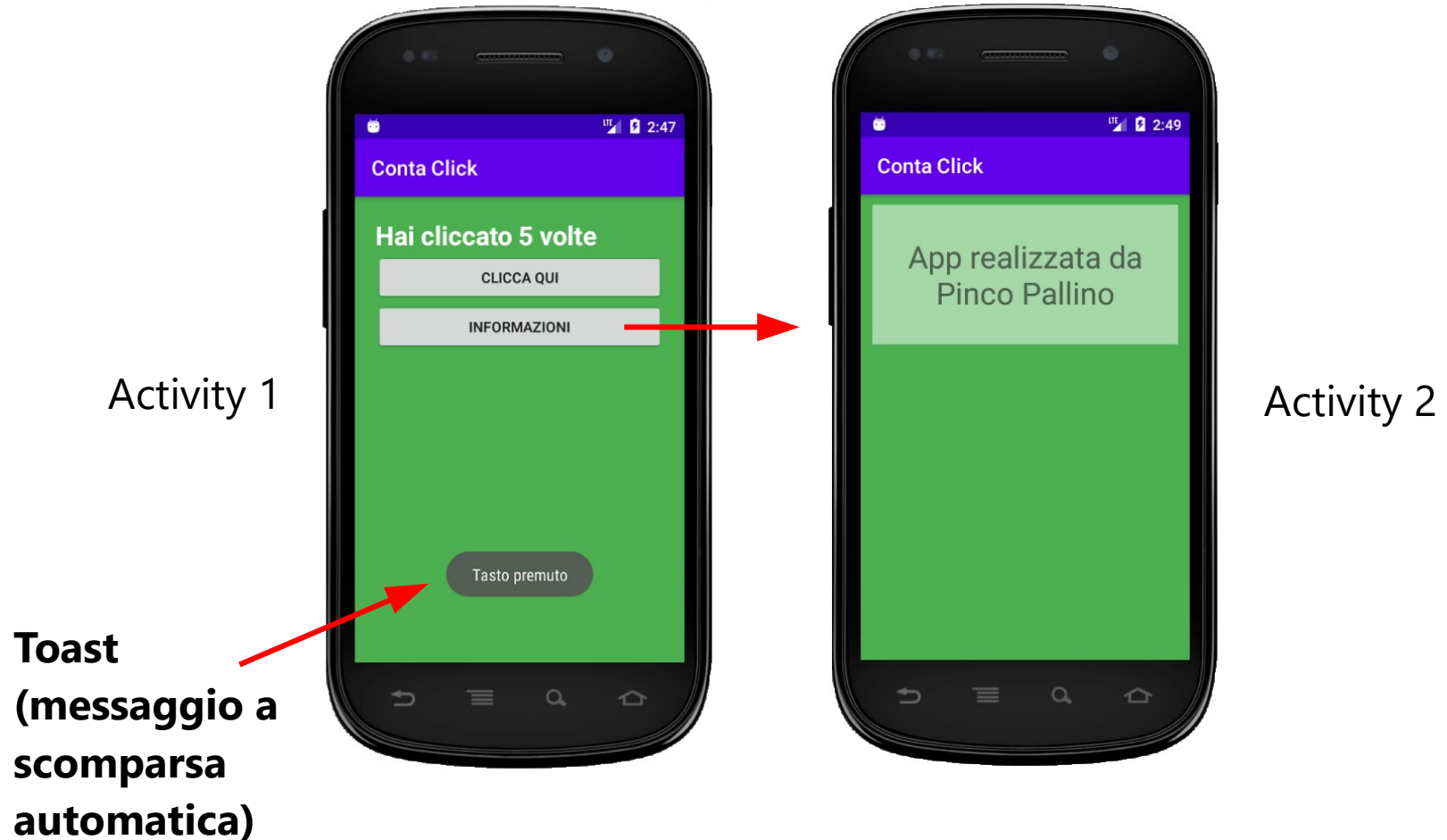
Programmazione Android

Toast e prima app

Introduzione

- Vediamo passo a passo come realizzare la nostra prima app in **Android Studio**
- Costruiremo un'app composta da due *schermate* (in Android le schermate sono chiamate **Activity**)
- Android Studio è l'ambiente di sviluppo (**IDE**) ufficiale per la creazione di app native per il sistema operativo Android

L'app di esempio

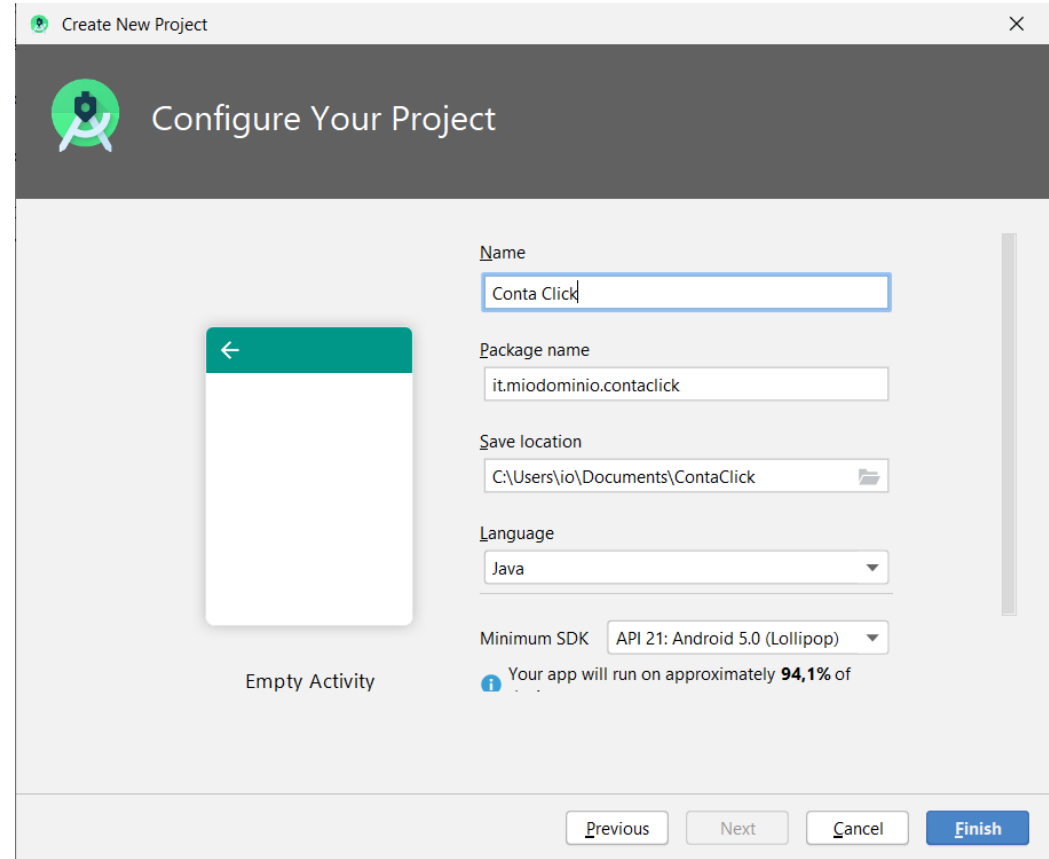


Argomenti

- Configurazione di un progetto
- Esecuzione dell'app su dispositivi virtuali (**AVD**) e fisici
- Struttura di un progetto in Android Studio (cartelle predefinite)
- File di layout (utilizzo dei tag **LinearLayout**, **Button**, **TextView**)
- Classi **Activity** e **Intent**
- Utilizzo di **Logcat** per il debug dell'app
- Generazione del file **APK** e distribuzione dell'app
- Esportazione/importazione di progetti in Android Studio

Creiamo il progetto

- Scegliere "Create New Project" e poi "Empty Views Activity"
- Impostare i campi come indicato dalla figura
- **nome** = il nome dell'app che l'utente vedrà sul telefono
- **package name** = stringa che identifica in modo univoco la mia app nel mondo. E' il "codice fiscale" della mia app.



Create New Project

Configure Your Project

Name: Conta Click

Package name: it.miodominio.contaclick

Save location: C:\Users\io\Documents\ContaClick

Language: Java

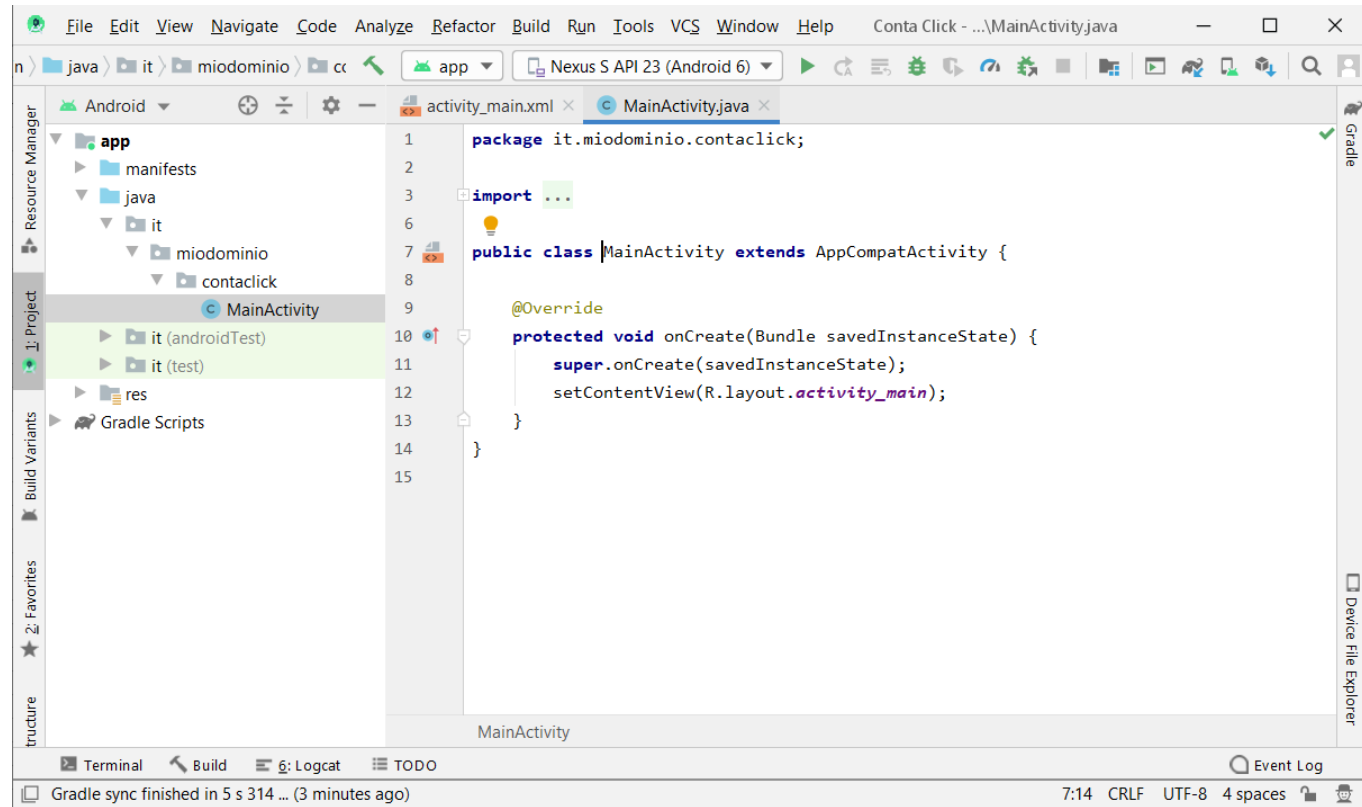
Minimum SDK: API 21: Android 5.0 (Lollipop)

Your app will run on approximately 94,1% of

Previous Next Cancel Finish

Creiamo il progetto

- **Minimum SDK** = minima versione di Android necessaria per installare l'app.
- Premendo "Finish", Android Studio genera un'app di esempio costituita da un'unica schermata (che mostra il testo "Hello World" !)



L'app "hello world" è completa e pronta per essere installata su un telefono virtuale o fisico

Osservazioni sul *package name*

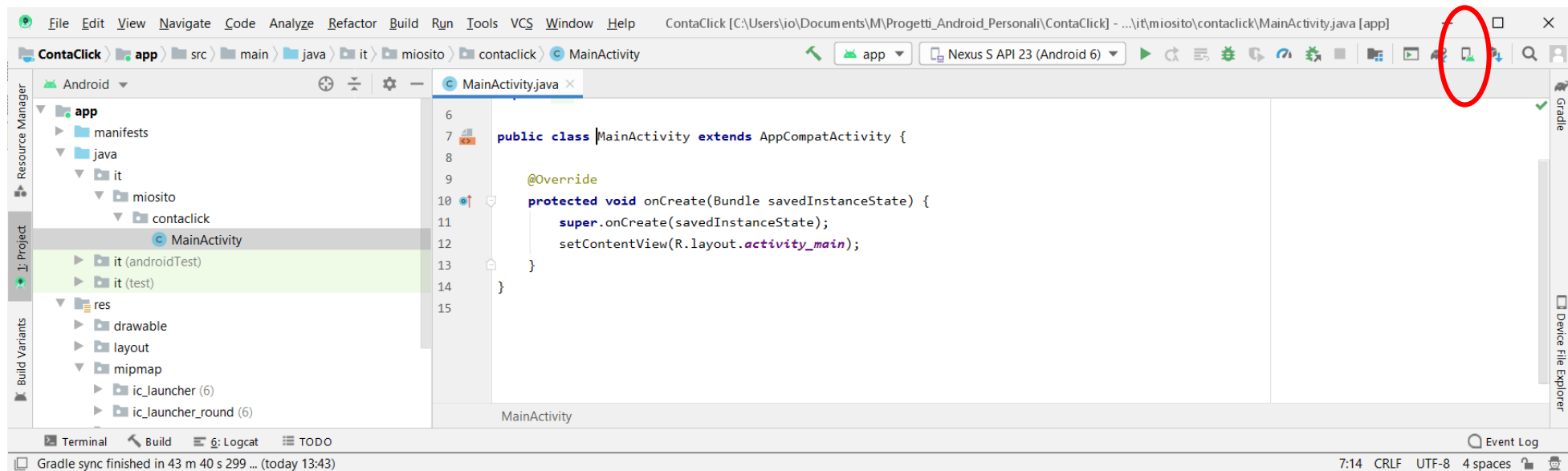
- Sul mercato possono esistere più app con lo stesso nome (si pensi a *Calcolatrice*) ma non dovrebbero esistere app con lo stesso "package name". Al fine di evitare ciò, per convenzione, gli sviluppatori usano un dominio web in loro possesso come parte iniziale del package. In questo modo solo il proprietario del dominio può usare legittimamente quel nome.
- Se non si possiede un dominio, se ne può usare uno fittizio (come *it.miodominio.nomeapp*) ma prima di pubblicare l'app sarà opportuno procurarsene uno, per il motivo seguente:
- il sistema operativo considera due app distinte se e solo se hanno un diverso package name. Se l'utente scarica un'app con lo stesso package name di una già installa, la seconda viene considerata un aggiornamento.

Osservazioni sul *minimum SDK*

- Abbassando il valore di questo parametro si supporta un maggior numero di dispositivi ma si rinuncia all'uso delle funzionalità più recenti e si rende quindi più difficile lo sviluppo.
- Per aiutare gli sviluppatori nella scelta, Android Studio mostra la distribuzione attuale delle versioni di Android tra i dispositivi attivi su Play Store.
- Ad esempio, se solo l'1% dei dispositivi attuali esegue una versione di Android inferiore a 5.0 potrebbe non essere redditizio supportare tali dispositivi.

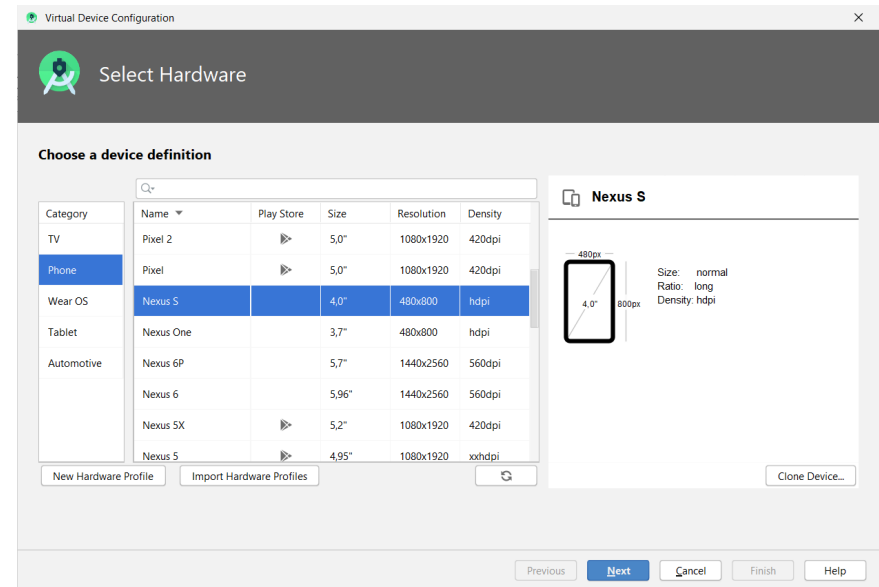
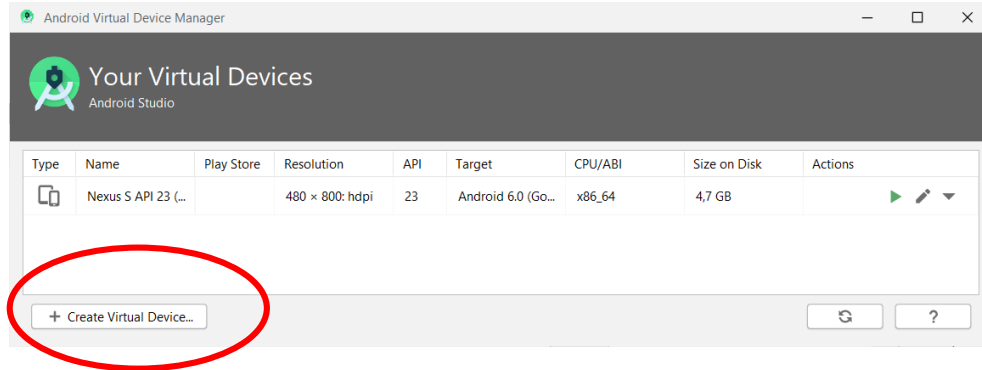
Esecuzione dell'app

- Per verificare che il progetto sia stato creato correttamente, proviamo ad eseguire l'app "hello world" in un **dispositivo virtuale** (**AVD = *Android Virtual Device***)
- Clicchiamo sull'icona evidenziata per aprire il gestore dei dispositivi virtuali (**AVD Manager**) e verifichiamo che sia presente almeno un dispositivo virtuale.



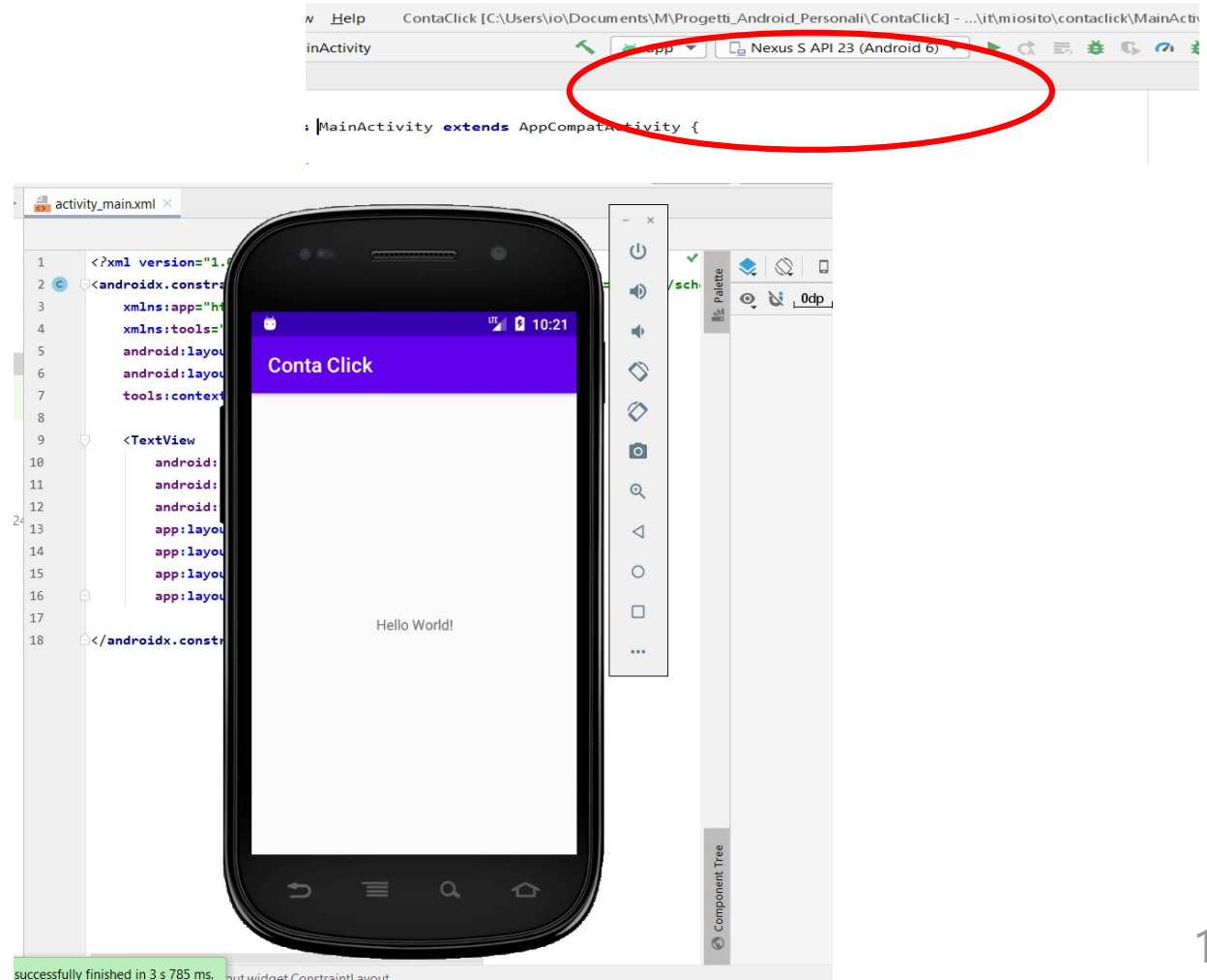
Creazione di un AVD

- Se non ci sono dispositivi occorre crearne uno scegliendo "Create Virtual Device"



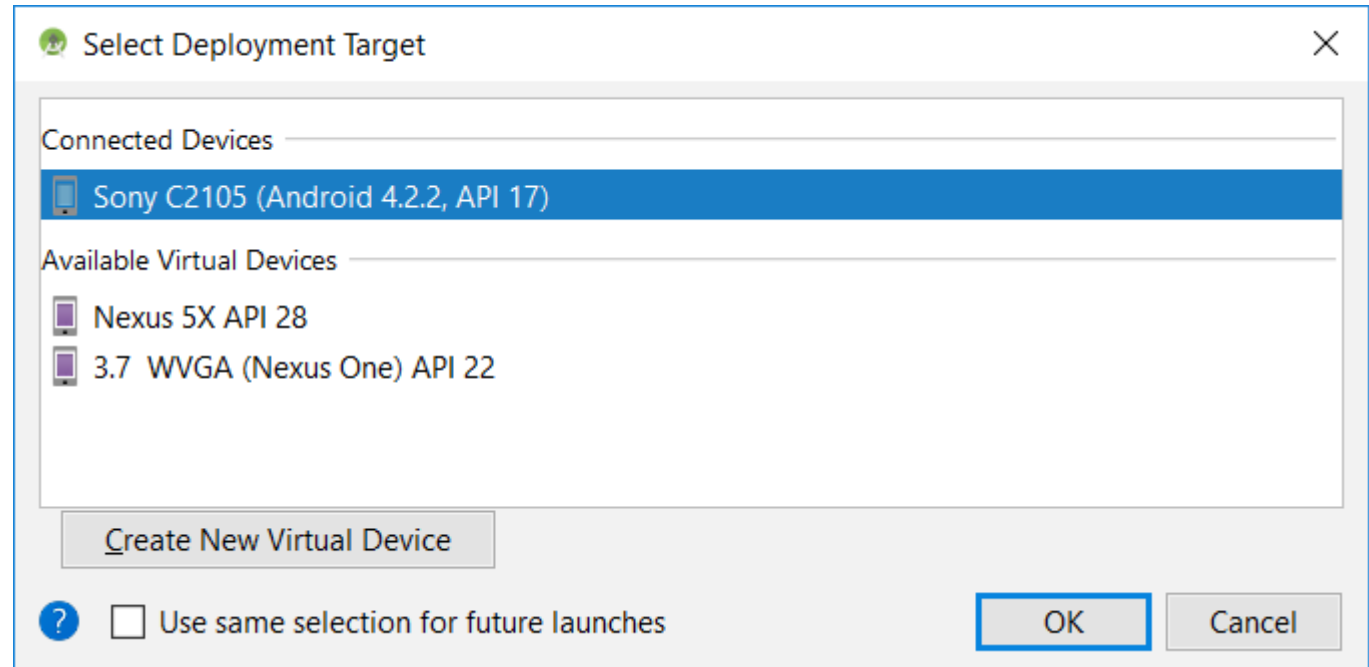
Esecuzione dell'app

- Per eseguire l'app selezioniamo dalla lista il dispositivo "target" (destinazione) e clicchiamo il tasto *run* (triangolo verde)

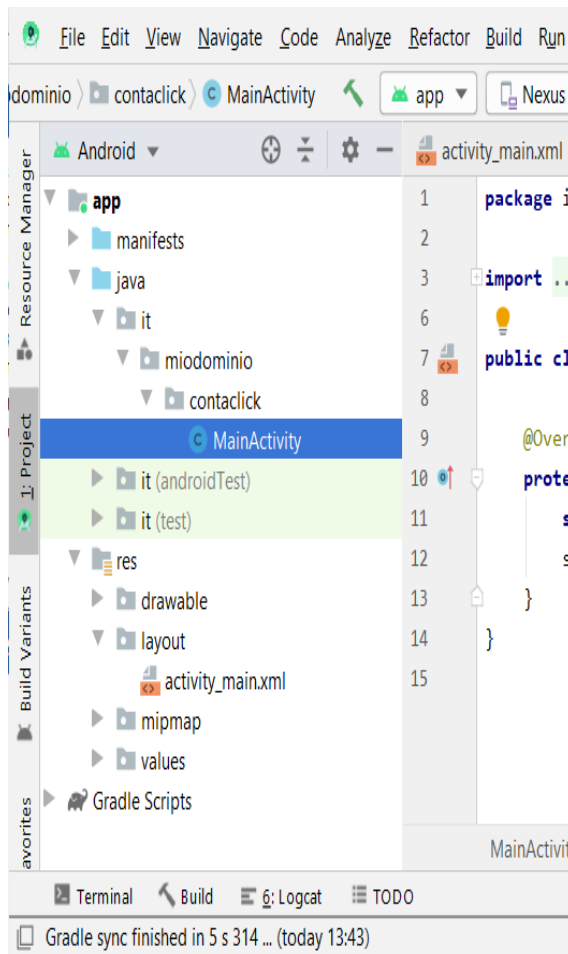


Esecuzione dell'app

- Nella lista dei dispositivi target compariranno sia gli AVD sia gli eventuali dispositivi fisici collegati al PC tramite cavo USB



Uno sguardo alla struttura del progetto



- Quando si crea un nuovo progetto, Android Studio genera numerosi file e li organizza in un **albero di cartelle con nomi standard (non modificabili)**
- Ogni cartella è destinata a contenere particolari **tipi di file** (*codice java, immagini, icone, stili, traduzioni ...*)
- Il programmatore deve conoscere questa struttura per posizionare correttamente i file.

Osservazioni sul file *manifest*

- la **cartella manifest** contiene il file *manifest.xml*

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="it.miodominio.contaclick">
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

File contenuto nella cartella res/mipmap

Nome dell'app specificato nella stringa *app_name* definita nel file strings.xml nella cartella res/values

Osservazioni sul file *manifest*

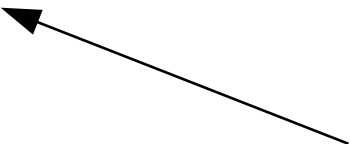
Il file *manifest.xml* è il file di configurazione principale dell'app. Con questo file l'app "si presenta" al sistema operativo, dichiarando ad Android:

- **nome del package** e **nome** dell'app
- **icone** di lancio (attributi `icon` e `roundIcon`)
- da quali **activity** è composta (in questo caso c'è una sola activity di nome `MainActivity`)
- quali **permessi** richiede, come *accedere a Internet*, *utilizzare Bluetooth*, *accedere ai contatti* (in questo caso non sono richiesti permessi)
- quali **azioni** sà svolgere, come *aprire una mappa*, *scattare una foto*, *inviare un messaggio* (in questo caso non sono specificati azioni)

Contenuto della cartella *java*

- La **cartella java** contiene le classi che compongono il progetto
- Androdi Studio ha generato qui un file di nome *MainActivity.java*, che rappresenta la schermata principale (l'unica presente).

```
package it.miodominio.contaclick;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```



identificatore del file
activity_main.xml che si trova
nella cartella /res/layout

Osservazioni sul file MainActivity.java

- In generale, ad ogni schermata dell'app corrisponde un file Java che estende la classe **Activity** (o una sua classe figlia come AppCompatActivity).
- La classe **Activity** è una delle classi fondamentali del framework Android, e rappresenta una schermata vuota
- Per aggiungere componenti grafici alla schermata il programmatore ridefinisce il metodo **onCreate** (cioè effettua l'*override* del metodo)
- All'interno di questo metodo, con l'istruzione **setContent**, si indica il nome di un **file di layout** che contiene l'elenco dei componenti da caricare nella schermata e la loro disposizione (layout).

Osservazioni sul file MainActivity.java

- setContentView è un metodo della classe Activity, e come tale è disponibile anche nella classe MainActivity (che lo eredita).
- R.layout.activity_main è una costante di tipo intero che "punta" al file *activity_main.xml* nella cartella res/layout
- Scorciatoia: da Android Studio è possibile aprire velocemente il file *activity_main.xml* cliccando sulla costante mentre si tiene premuto il tasto CTRL.

Contenuto della cartella *res*

- La **cartella res** contiene le altre **risorse dell'app** organizzate in sottocartelle:

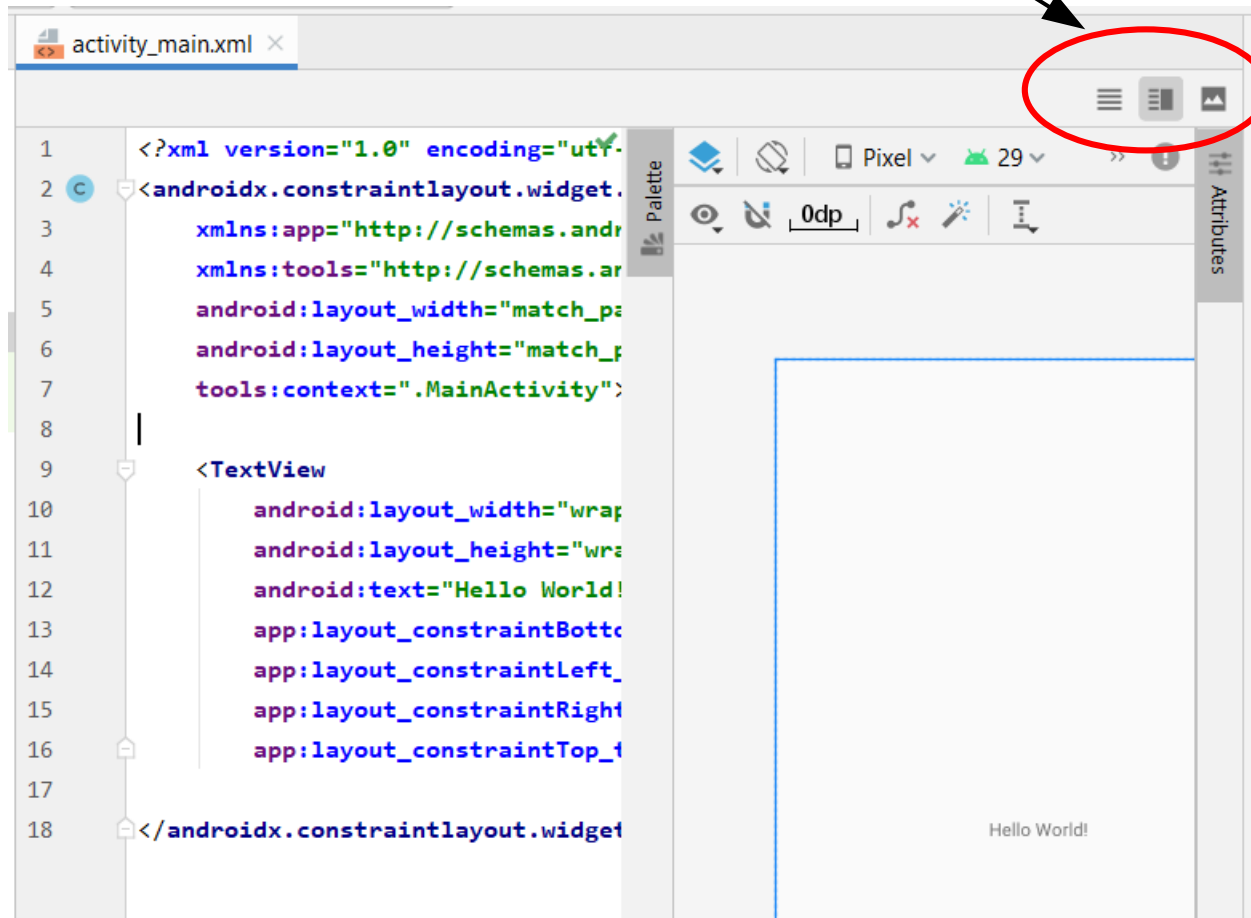
Cartella	Contenuto
<i>drawable</i>	immagini, icone, sfondi
<i>layout</i>	file di layout, ovvero file xml che descrivono l'aspetto grafico delle schermate o di loro parti
<i>mipmap</i>	icone di lancio dell'applicazione
<i>values</i>	file xml che contengono informazioni su: - colori, stili - traduzione dell'app in altre lingue

Visualizzazione dei file di layout

Un file di layout può essere visualizzato in tre modalità:

- **testuale** (**Code**): viene mostrato il codice xml
- **grafica** (**Design**): viene mostrata l'anteprima del file con possibilità di trascinare e configurare componenti
- **mista** (**Split**): viene mostrato sia il codice che l'anteprima

Pulsanti per cambiare visualizzazione



Il file *activity_main.xml*

- Un **file di layout** è un file xml che descrive l'aspetto grafico di una schermata o di una sua parte (analogo ad HTML)
- Specifica la disposizione (layout) dei componenti nella pagina...

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Aggiunta di un toast

Aggiunta di un toast

**Toast
(messaggio a
scomparsa
automatica)**



- E' un modo per comunicare dati in output sul display.
- Toast è una classe che mette a disposizione dei messaggi a scomparsa

@Override

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    EdgeToEdge.enable( $this$enableEdgeToEdge: this);  
    setContentView(R.layout.activity_main);  
    int tasto=1;  
    Toast toast=Toast.makeText( context: this, text: "tasto premuto "+tasto, Toast.LENGTH_LONG);  
    toast.show();  
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {  
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());  
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);  
        return insets;  
    });  
}
```

Metodo makeText
Vuole 3 parametri:
- Il contesto: this
- il testo da visualizzare
- la durata del messaggio (2 costanti):
LENGTH_LONG
LENGTH_SHORT

- Classe Toast
- Metodo makeText
- Metodo show

ESERCIZIO

- Crea un'applicazione in grado di generare 10 messaggi brevi di tipo Toast che visualizzani i primi 10 numeri primi.